Lecture 6 - Comprehensive Study of Trees

6.1 Binary Trees

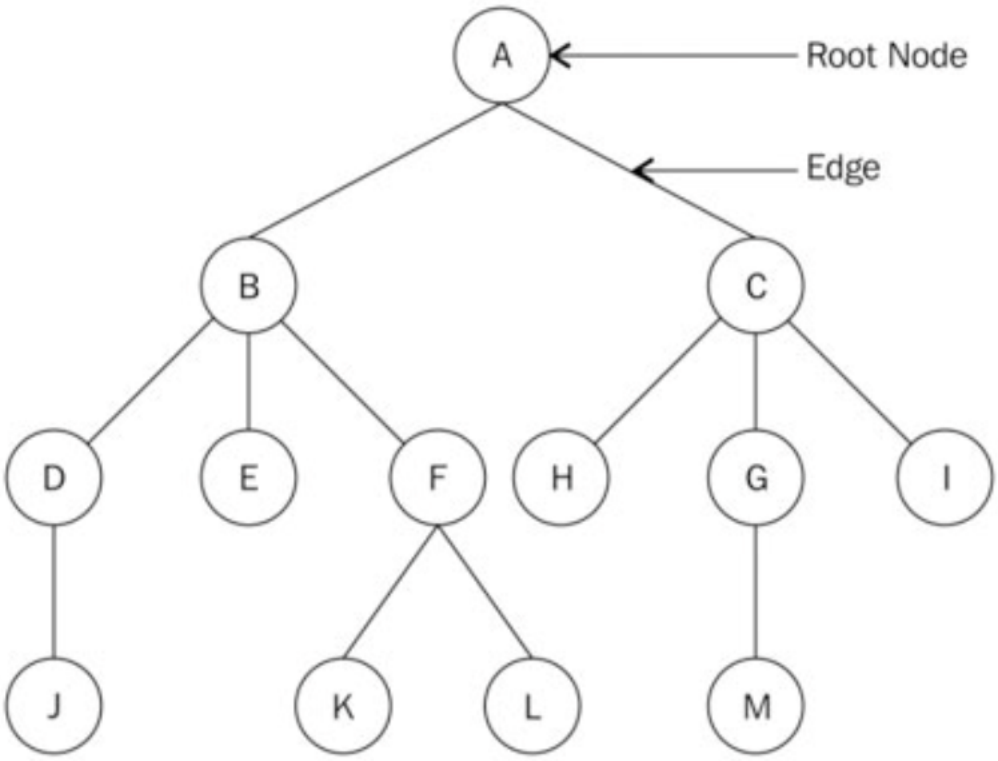
6.2 Binary Search Tree

6.3 Python: Binary Search Trees

6.4 AVL Trees

6.1 Binary Trees

A tree is a hierarchical form of data structure. When we dealt with lists, queues, and stacks, items followed each other. In a tree, there is a parent-child relationship between items. Trees are normally drawn downward. At the top of every tree is the root node. This is the ancestor of all other nodes in the tree.



CSU Global, 2020

Let's get started by reviewing the following introduction to the tree data structure:

Node

Each circled alphabet represents a node. A node is any structure that holds data.

Root node

The root node is the only node from which all other nodes come. A tree with an undistinguishable root node cannot be considered a tree. The root node in the above example is the node A.

Sub-tree

A sub-tree of a tree is a tree with its nodes being a descendant of some other tree. Nodes F, K, and L form a sub-tree of the original tree consisting of all the nodes.

Degree

The number of sub-trees of a given node. A tree consisting of only one node has a degree of 0. This one tree node is also considered a tree by all standards. The degree of node A is 2.

Leaf node

This is a node with a degree of 0. Nodes J, E, K, L, H, M, and I are all leaf nodes.

Edge

The connection between two nodes. An edge can sometimes connect a node to itself, making the edge appear as a loop.

Parent

A node in the tree with other connecting nodes is the parent of those nodes. Node B is the parent of nodes D, E, and F.

Child

This is a node connected to its parent. Nodes B and C are children of node A, the parent and root node.

Sibling

All nodes with the same parent are siblings. This makes the nodes B and C siblings.

Level

The level of a node is the number of connections from the root node. The root node is at level 0. Nodes B and C are at level 1.

Height of a tree

This is the number of levels in a tree. Our tree has a height of 4.

Depth

The depth of a node is the number of edges from the root of the tree to that node. The depth of node H is 2.

Trees are built up of nodes. The nodes that make up a tree need to contain data about the parent-child relationship. Let us review this example of how to build a binary tree node class in Python:

```
       def __init__(self, data):
            self.data = data
            self.right_child = None
            self.left_child = None
```

A node is a container for data and holds references to other nodes. Being a binary tree node, these references are to the left and the right children. To test this class out, we first create a few nodes:

```
    n1 = Node("root node")
    n2 = Node("left child node")
    n3 = Node("right child node")
    n4 = Node("left grandchild node")
```

Next, we connect the nodes to each other. We let n1 be the root node with n2 and n3 as its children. Finally, we hook n4 as the left child to n2, so that we get a few iterations when we traverse the left sub-tree:

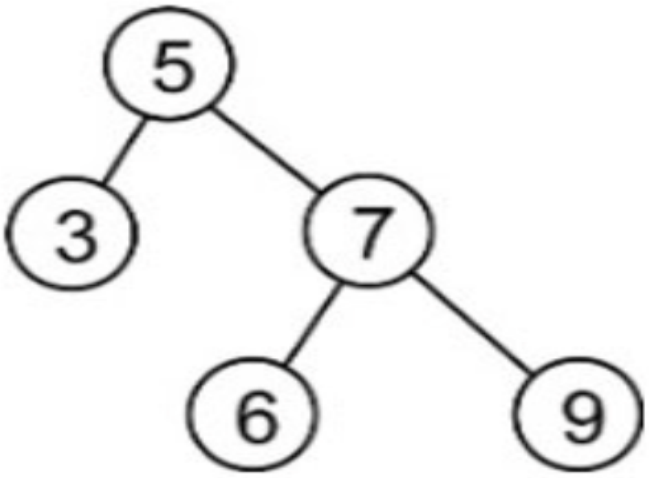
```
    n1.left_child = n2
    n1.right_child = n3
    n2.left_child = n4
```

Once we have our tree structure set up, we are ready to traverse it. We will traverse the left sub-tree. We print out the node and move down the tree to the next left node. We keep doing this until we have reached the end of the left sub-tree:

```
    current = n1
    while current:
        print(current.data)
        current = current.left_child
```

This requires quite a bit of work in the client code, as you have to manually build up the tree structure. A binary tree is one in which each node has a maximum of two children. Binary trees are very common, and we will be using them for BST implementation in Python, which we will discuss next.

The following figure is an example of a binary tree with 5 being the root node:

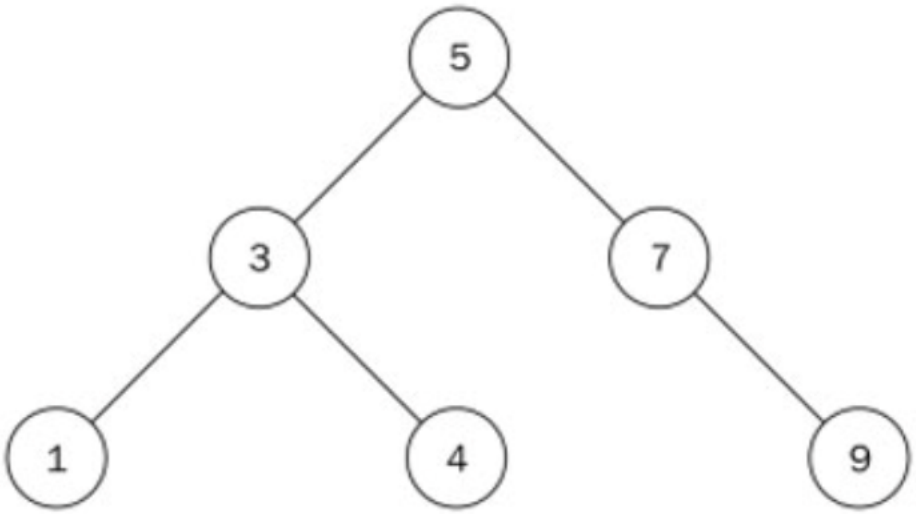


CSU Global, 2020

Each child is identified as being the right or left child of its parent. Since the parent node is also a node by itself, each node will hold a reference to a right and left node even if the nodes do not exist. A regular binary tree has no rules as to how elements are arranged in the tree. It only satisfies the condition that each node should have a maximum of two children.

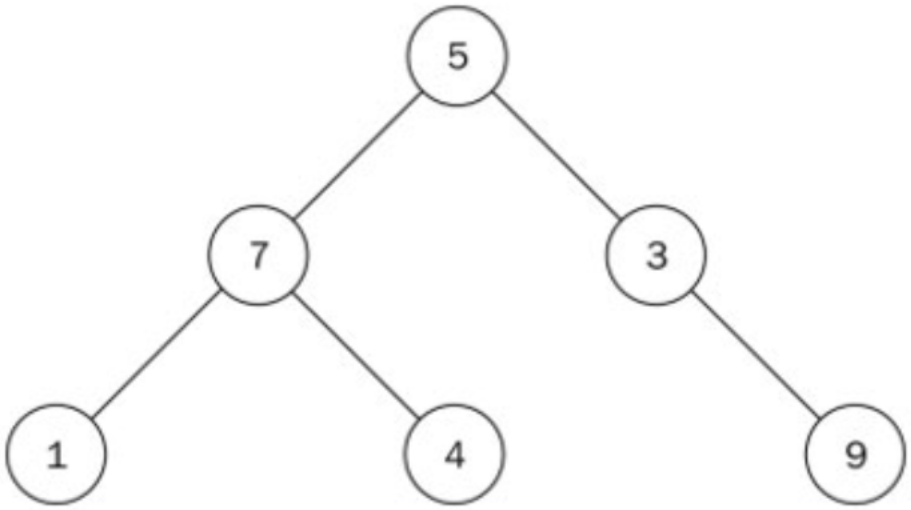
6.2 Binary Search Tree

A binary search tree (BST) is structurally a binary tree but functionally, it is a tree that stores its nodes in a way to efficiently search through the tree. There is a structure to a BST. For a given node with a value, all the nodes in the left sub-tree are less than or equal to the value of that node. In addition, all the nodes in the right sub-tree of this node are greater than that of the parent node. As an example, consider the following tree:



CSU Global, 2020

This is an example of a BST. All the nodes in the left sub-tree of the root node have a value less than 5. Likewise, all the nodes in the right sub-tree have a value that is greater than 5. This property applies to all the nodes in a BST, with no exceptions:



CSU Global, 2020

Despite the fact that the preceding figure looks similar to the previous figure, it does not qualify as a BST. Node 7 is greater than the root node 5; however, it is located to the left of the root node. Node 4 is to the right sub-tree of its parent node 7, which is incorrect.

Before continuing, please watch this video on BSTs: **Binary Search Tree Concept**

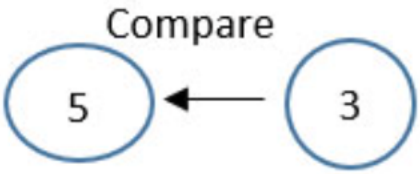
To begin our implementation of a BST, we will want the tree to hold a reference to its own root node:

```
class Tree:
    def __init__(self):
        self.root_node = None
```

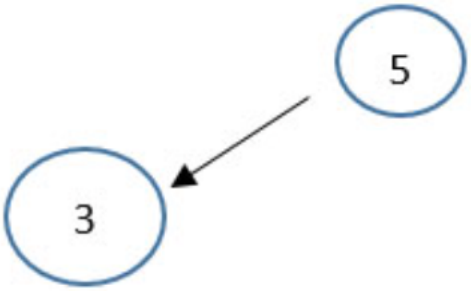
There are essentially two operations that are needed for a usable BST. These are the *insert* and *remove* operations. These operations must occur with the one rule that they must maintain the principle that gives the BST its structure.

Inserting Nodes

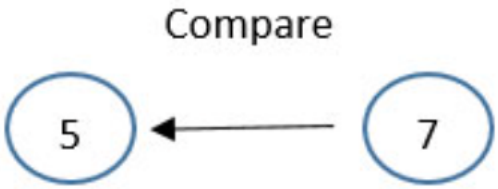
In order to make a search possible, the nodes must be stored in a specific way. For each given node, its left child node will hold data that is less than its own value. That node's right child node will hold data greater than that of its parent node. We are going to create a new BST of integers by starting with the data 5. To do this, we will create a node with its data attribute set to 5. To add the second node with value 3, 3 is compared with 5, the root node:



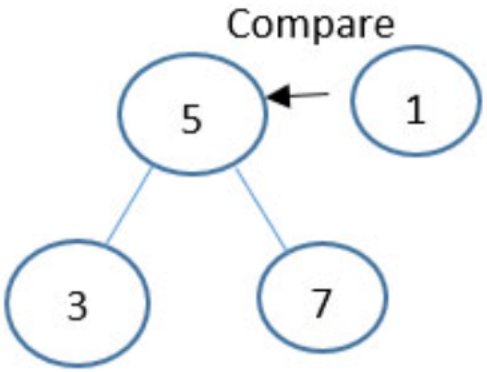
Since 5 is greater than 3, it will be put in the left sub-tree of node 5. Our BST will look as follows:



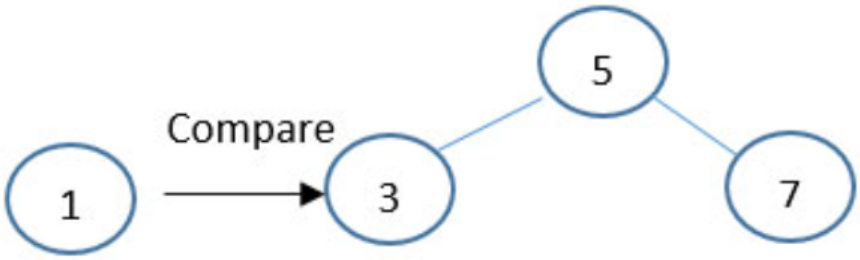
The tree satisfies the BST rule, where all the nodes in the left sub-tree are less than its parent. To add another node of value 7 to the tree, we start from the root node with value 5 and do a comparison:



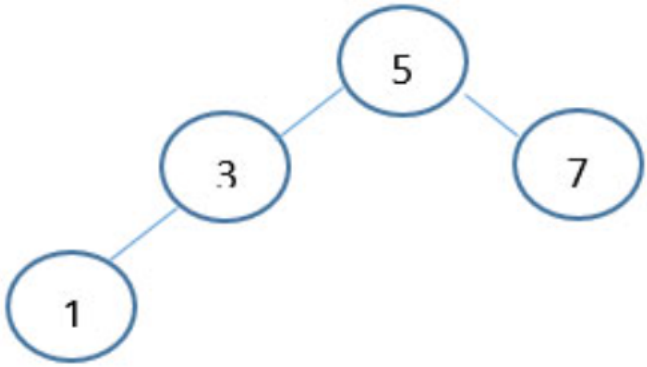
Since 7 is greater than 5, the node with value 7 is situated to the right of this root. What happens when we want to add a node that is equal to an existing node? We will just add it as a left node and maintain this rule throughout the structure. If a node already has a child in the place where the new node goes, then we have to move down the tree and attach it. Let's add another node with value 1. Starting from the root of the tree, we do a comparison between 1 and 5:



The comparison reveals that 1 is less than 5, so we move onto the left node of 5, which is the node with value 3:



We compare 1 with 3 and since 1 is less than 3, we move a level below node 3 and to its left. However, there is no node there. Therefore, we create a node with the value 1 and associate it with the left pointer of node 3 to obtain the following structure:



So far, we have been dealing only with nodes that contain only integers or numbers. For numbers, the idea of greater than and lesser than are clearly defined. Strings would be compared alphabetically, so there are no major problems there either. However, if you

want to store your own custom data types inside a BST, you would have to make sure that your class supports ordering.

Let's now create a function that enables us to add data as nodes to the BST. We begin with a function declaration:

```
   def insert(self, data):
```

We encapsulate the data in a node. This way, we hide away the node class from the client code, who only needs to deal with the tree:

```
   node = Node(data)
```

A first check will be to find out whether we have a root node. If we don't, the new node becomes the root node (we cannot have a tree without a root node):

```
   if self.root_node is None:
   self.root_node = node
   else:
```

As we walk down the tree, we need to keep track of the current node we are working on, as well as its parent. The variable current is always used for this purpose:

```
   current = self.root_node
   parent = None
   while True:
   parent = current
```

Here we must perform a comparison. If the data held in the new node is less than the data held in the current node, then we check whether the current node has a left child node. If it doesn't, this is where we insert the new node. Otherwise, we keep traversing:

```
   if node.data
```

Now we take care of the greater than or equal case. If the current node doesn't have a right child node, then the new node is inserted as the right child node. Otherwise, we move down and continue looking for an insertion point:

```
else:
   current = current.right_child
   if current is None:
   parent.right_child = node
   return
```

Insertion of a node in a BST takes **O**(*h*), where h is the height of the tree.

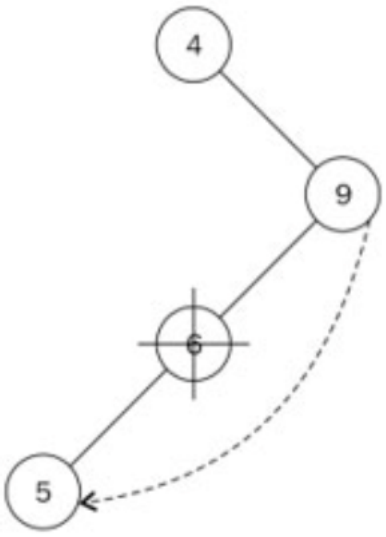
Deleting Nodes

The node that we want to remove might have the following:

- No children
- One child
- Two children

The first scenario is the easiest to handle. If the node about to be removed has no children, we simply detach it from its parent:

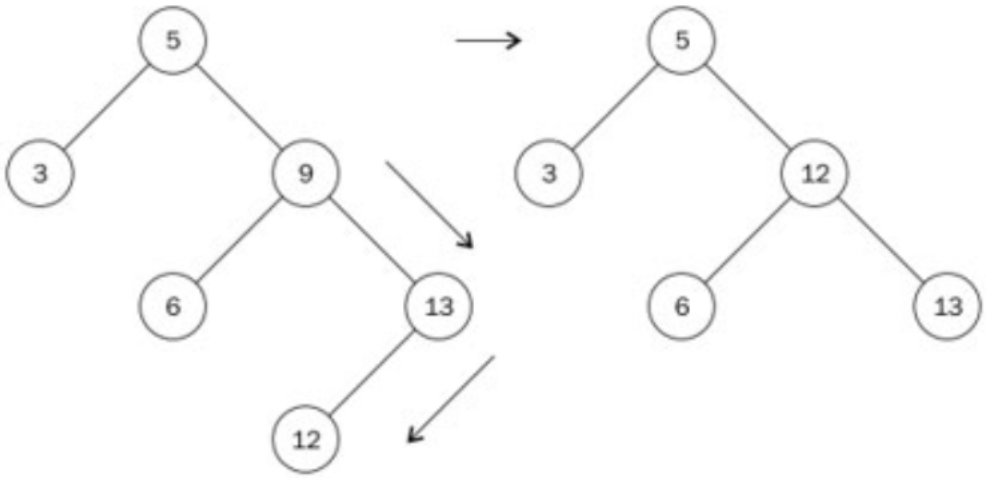
Because node A has no children, we will simply dissociate it from its parent, node Z. On the other hand, when the node we want to remove has one child, the parent of that node is made to point to the child of that particular node:



In order to remove node 6, which has as its only child node 5, we point the left pointer of node 9 to node 5. The relationship between the parent node and child has to be preserved. That is why we need to take note of how the child node is connected to its

parent (which is the node about to be deleted). The child node of the deleted node is stored. Then we connect the parent of the deleted node to that child node.

A more complex scenario arises when the node we want to delete has two children:



We cannot simply replace node 9 with either node 6 or 13. What we need to do is to find the next biggest descendant of node 9. This is node 12. To get to node 12, we move to the right node of node 9. Then, move left to find the leftmost node. Node 12 is called the in-order successor of node 9. The second step resembles the move to find the maximum node in a sub-tree. We replace the value of node 9 with the value 12 and remove node 12. In removing node 12, we end up with a simpler form of node removal that has been addressed previously. Node 12 has no children, so we apply the rule for removing nodes without children accordingly.

Our node class does not have reference to a parent. As such, we need to use a helper method to search for and return the node with its parent node. This method is similar to the search method:

```
def get_node_with_parent(self, data):
parent = None
current = self.root_node
if current is None:
return (parent, None)
while True:
if current.data == data:
return (parent, current)
elif current.data  data:
parent = current
current = current.left_child
else:
parent = current
```

```
current = current.right_child
return (parent, current)
```

The only difference is that before we update the current variable inside the loop, we store its parent with parent = current. The method to do the actual removal of a node begins with this search:

```
def remove(self, data):
parent, node = self.get_node_with_parent(data)
if parent is None and node is None:
return False
# Get children count
children_count = 0
if node.left_child and node.right_child:
children_count = 2
elif (node.left_child is None) and (node.right_child is None):
children_count = 0
else:
children_count = 1
```

We pass the parent and the found node to parent and node respectively with the line parent, node = self.get_node_with_parent(data). It is helpful to know the number of children that the node we want to delete has. That is the purpose of the if statement. After this, we need to begin handling the various conditions under which a node can be deleted. The first part of the if statement handles the case where the node has no children:

```
if children_count == 0:
if parent:
if parent.right_child is node:
parent.right_child = None
else:
parent.left_child = None
else:
self.root_node = None
```

NOTE: if parent: is used to handle cases where there is a BST that has only one node in the whole of the three.

In the case where the node about to be deleted has only one child, the *elif* part of the if statement does the following:

```
elif children_count == 1:
next_node = None
if node.left_child:
next_node = node.left_child
```

```
else:
next_node = node.right_child
if parent:
if parent.left_child is node:
parent.left_child = next_node
else:
parent.right_child = next_node
else:
self.root_node = next_node
```

NOTE: next_node is used to keep track of where the single node pointed to by the node we want to delete is. We then connect parent.left_child or parent.right_child to next_node.

Lastly, we handle the condition where the node we want to delete has two children:

```
...
else:
parent_of_leftmost_node = node
leftmost_node = node.right_child
while leftmost_node.left_child:
parent_of_leftmost_node = leftmost_node
leftmost_node = leftmost_node.left_child
node.data = leftmost_node.data
```

In finding the in-order successor, we move to the right node with *leftmost_node = node.right_child*. As long as there exists a left node, *leftmost_node.left_child* will evaluate to True and the while loop will run. When we get to the leftmost node, it will either be a leaf node (meaning that it will have no child node) or have a right child.

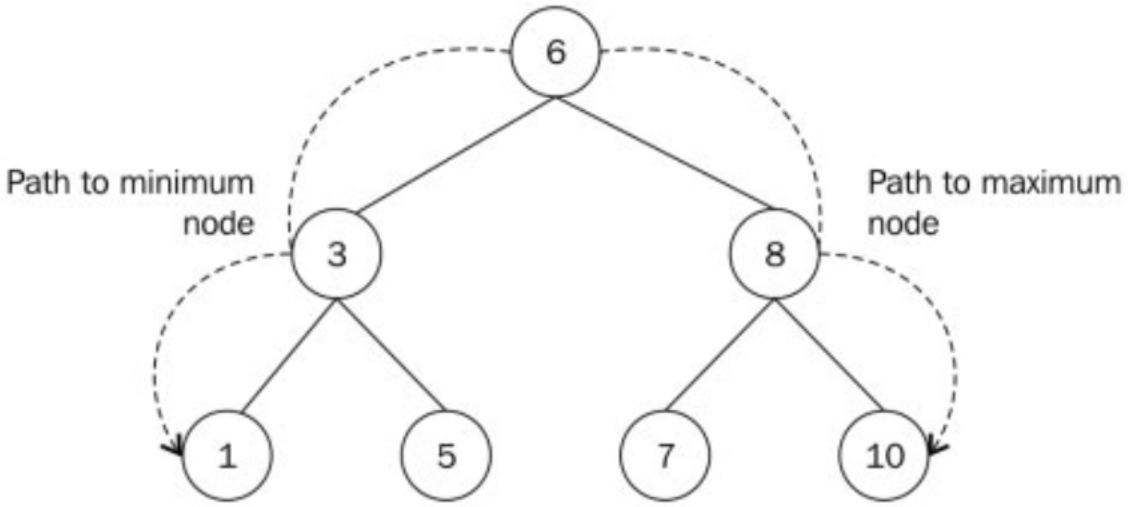
We update the node about to be removed with the value of the in-order successor with *node.data = leftmost_node.data*:

```
if parent_of_leftmost_node.left_child == leftmost_node:
parent_of_leftmost_node.left_child = leftmost_node.right_child
else:
parent_of_leftmost_node.right_child = leftmost_node.right_child
```

The preceding statement allows us to properly attach the parent of the leftmost node with any child node. The right-hand side of the equal sign stays unchanged. That is because the in-order successor can only have a right child as its only child. The remove operation takes **O**(*h*), where *h* is the height of the tree.

Finding Min and Max Nodes

To find the node with smallest value, we start our traversal from the root of the tree and visit the left node each time we reach a sub-tree. We do the opposite to find the node with the biggest value in the tree:



CSU Global, 2020

We move down from node 6 to 3 to 1 to get to the node with smallest value. Likewise, we go down 6, 8 to node 10, which is the node with the largest value. This same means of finding the minimum and maximum nodes applies to sub-trees too. The minimum node in the sub-tree with root node 8 is 7. The node within that sub-tree with the maximum value is 10.

The method that returns the minimum node is as follows:

```
    def find_min(self):
        current = self.root_node
        while current.left_child:
            current = current.left_child
        return current
```

The while loop continues to get the left node and visits it until the last left node points to None. The method to return the maximum node does the opposite, where *current.left_child* now becomes *current.right_child*. It takes **O***(h)* to find the minimum or maximum value in a BST, where *h* is the height of the tree.

6.3 Python: Binary Search Trees

Inorder Traversals

Due to the binary search tree property, we can run the following algorithm, called **inorder traversal**, where the result is going to be the list of all the node values of the tree sorted in ascending order. The algorithm will use recursion to visit each subtree of the root node in the following order: First the left subtree—then the node value—then the right subtree.

Knowing that in a BST: ***Left value < node value < right value***

This will cause a sequence of ascending ordered values as the output. **Example pseudocode:**

```
//In-order traversal with recursion
    public class func traverseInOrder(node:BinaryTreeNode?) {
          // The recursive calls end when we reach a Nil leaf
        guard let node = node else {
            return
        }
        // Recursively call the method again with the leftChild,
        // then print the value, then with the rigthChild
        BinaryTreeNode.traverseInOrder(node: node.leftChild)
        print(node.value)
        BinaryTreeNode.traverseInOrder(node: node.rightChild)
    }
```

BST Height

The height of a position ***p*** in a tree ***T*** is defined recursively:

- If ***p*** is a leaf, then the height of ***p*** is 0.
- Otherwise, the height of ***p*** is one more than the maximum of the heights of ***p***'s children.

The height of a nonempty tree ***T*** is the height of the root of ***T***. In addition, height can also be viewed as:

- The height of a nonempty tree ***T*** is equal to the maximum of the depths of its leaf positions.

Please review this video for more information on the height of a binary tree.

Python Code Example 1:

```
Algorithm after(p):
if right(p) is not None then {successor is leftmost position in p's right sub
tree}
walk = right(p)
while left(walk) is not None do
walk = left(walk)
return walk
else {successor is nearest ancestor having p in its left subtree}
walk = p
ancestor = parent(walk)
while ancestor is not None and walk == right(ancestor) do
walk = ancestor
ancestor = parent(walk)
return ancestor
Python Code Fragment: Computing the successor of a position in a binary searc
h tree.
```

Python Code Example 2:

```
Algorithm TreeSearch(T, p, k):
if k == p.key() then
return p                    {successful search}
else if k  p.key() and T.right(p) is not None then
return TreeSearch(T, T.right(p), k)        {recur on right subtree}
return p                     {unsuccessful search}
Python Code Fragment: Recursive search in a binary search tree.
```

6.4 AVL Trees

Any binary search tree ***T*** that satisfies the height-balance property is said to be an AVL tree, named after the initials of its inventors: Adel'son-Vel'skii and Landis.

Given a binary search tree ***T***, we say that a position is balanced if the absolute value of the difference between the heights of its children is at most 1, and we say that it is unbalanced otherwise. Thus, the height-balance property characterizing AVL trees is equivalent to saying that every position is balanced.

The insertion and deletion operations for AVL trees begin similarly to the corresponding operations for (standard) binary search trees, but with post-processing for each operation to restore the balance of any portions of the tree that are adversely affected by the change. AVL trees are strictly balanced trees and inserting a new node can break that balance. Once we put the new node in the correct subtree, we need to ensure that the balance factors of its ancestors are correct. This process is called retracing. If there is any balance factor with wrong values (not in the range [-1 , 1]), we will use rotations to fix it.

Deletion from a regular binary search tree results in the structural removal of a node having either zero or one children. Such a change may violate the height-balance property in an AVL tree. Particularly, if position p represents the parent of the removed node in tree ***T***, there may be an unbalanced node on the path from p to the root of ***T***.

Class AVLTreeMap(TreeMap):

```
#------------------------- nested Node class -------------------------
class _Node(TreeMap. Node):
__slots__    =    '_height'        # additional data member to store height
def __init__(self, element, parent=None, left=None, right=None):
super().    __init__(element, parent, left, right)
self._height = 0     # will be recomputed during balancing
def left_height(self):
return self._left._height if self._left is not None else 0
def right_height(self):
return self._right._height if self._right is not None else 0
Python Code Fragment: AVLTreeMap class
```

Although AVL trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion.

Summary

This module focused on binary search trees and AVL trees. Next week, we will be discussing graphs. With all the content we've covered the past six weeks, you are ready for our thorough discussion of graphs next week. As you learned here in Module 6, tree structures allow us to implement a host of algorithms much faster than when using linear data structures. A tree is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a parent element and zero or more children elements. Binary search tree basic operations such as access, search, insertion, and deletion take between ***O(n)*** and ***O(log(n))*** time. Being both values the worst and the average scenarios. At the end, these times are going to depend on the height of the tree itself.

To practice and review the skills you will need to complete future activities, work through these labs in Zybooks ***CSC506: Design and Analysis of Algorithms***:

- 6.22 LAB: BST traversal
- 6.23 LAB: AVL tree vs. BST tree

Additionally, there are some optional sections in Zybooks on **red-black tree**. Although these sections are optional, this is also an important concept to understand before moving onto Modules 7 and 8.